

Counting Sort

计数排序

▼ 提醒

本页面要介绍的不是 [基数排序](#)。

本页面将简要介绍计数排序。

定义

计数排序（英语：Counting sort）是一种线性时间的排序算法。

过程

计数排序的工作原理是使用一个额外的数组，其中第 i 个元素是待排序数组中值等于 i 的元素的个数，然后根据数组来将 A 中的元素排到正确的位置。^{[1](#)}

它的工作过程分为三个步骤：

1. 计算每个数出现了几次；
2. 求出每个数出现次数的 [前缀和](#)；
3. 利用出现次数的前缀和，从右至左计算每个数的排名。

计算前缀和的原因

阅读本章内容只需要了解前缀和概念即可

直接将 A 中正数对应的元素依次放入 B 中不能解决元素重复的情形。

我们通过为额外数组 C 中的每一项计算前缀和，结合每一项的数值，就可以为重复元素确定一个唯一排名：

额外数组 C 中每一项的数值即是该 key 值下重复元素的个数，而该项的前缀和即是排在最后一个的重复元素的排名。

如果按照 C 的逆序进行排列，那么显然排序后的数组将保持 A 的原序（相同 key 值情况下），也即得到一种稳定的排序算法。

2	2	5	A	3	A	4	A	4	5	
										5
										5

性质

稳定性

计数排序是一种稳定的排序算法。

时间复杂度

计数排序的时间复杂度为 $O(n+k)$ ，其中 k 代表待排序数据的值域大小。

代码实现

伪代码

```

1 #include <LTcstring>
2 constexpr int MAXN = 1010;
3 constexpr int MAXW = 100010;
4
5 int cnt[MAXW], b[MAXN];
6
7 int* counting_sort(int* a, int n, int w) {
8     memset(cnt, 0, sizeof(cnt));
9     for (int i = 1; i <= n; ++i) ++cnt[a[i]];
10    for (int i = 1; i <= w; ++i) cnt[i] += cnt[i - 1];
11    for (int i = n; i >= 1; --i) b[cnt[a[i]]--] = a[i];
12    return b;
13 }

```

```

1 N = W = 100010
2 n = w = 0
3 a = b = [0] * N
4 cnt = [0] * W
5
6
7 def counting_sort():
8     for i in range(1, n + 1):
9         cnt[a[i]] += 1
10    for i in range(1, w + 1):
11        cnt[i] += cnt[i - 1]
12    for i in range(n, 0, -1):
13        b[cnt[a[i]] - 1] = a[i]
14        cnt[a[i]] -= 1

```

参考资料与注释

1. [计数排序 - 维基百科，自由的百科全书](#) ↗

本页面最近更新：2024/11/10 20:22:04, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [iamtwz](#), [NachtgeistW](#), [Alisahhh](#), [Enter-tainer](#), [gi-b716](#), [Great-designer](#), [Junyan721113](#), [Konano](#), [ksyx](#), [mcendu](#), [Menci](#), [minghu6](#), [ouuan](#), [shawllew](#), [Tiphereth-A](#), [TrickEye](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用